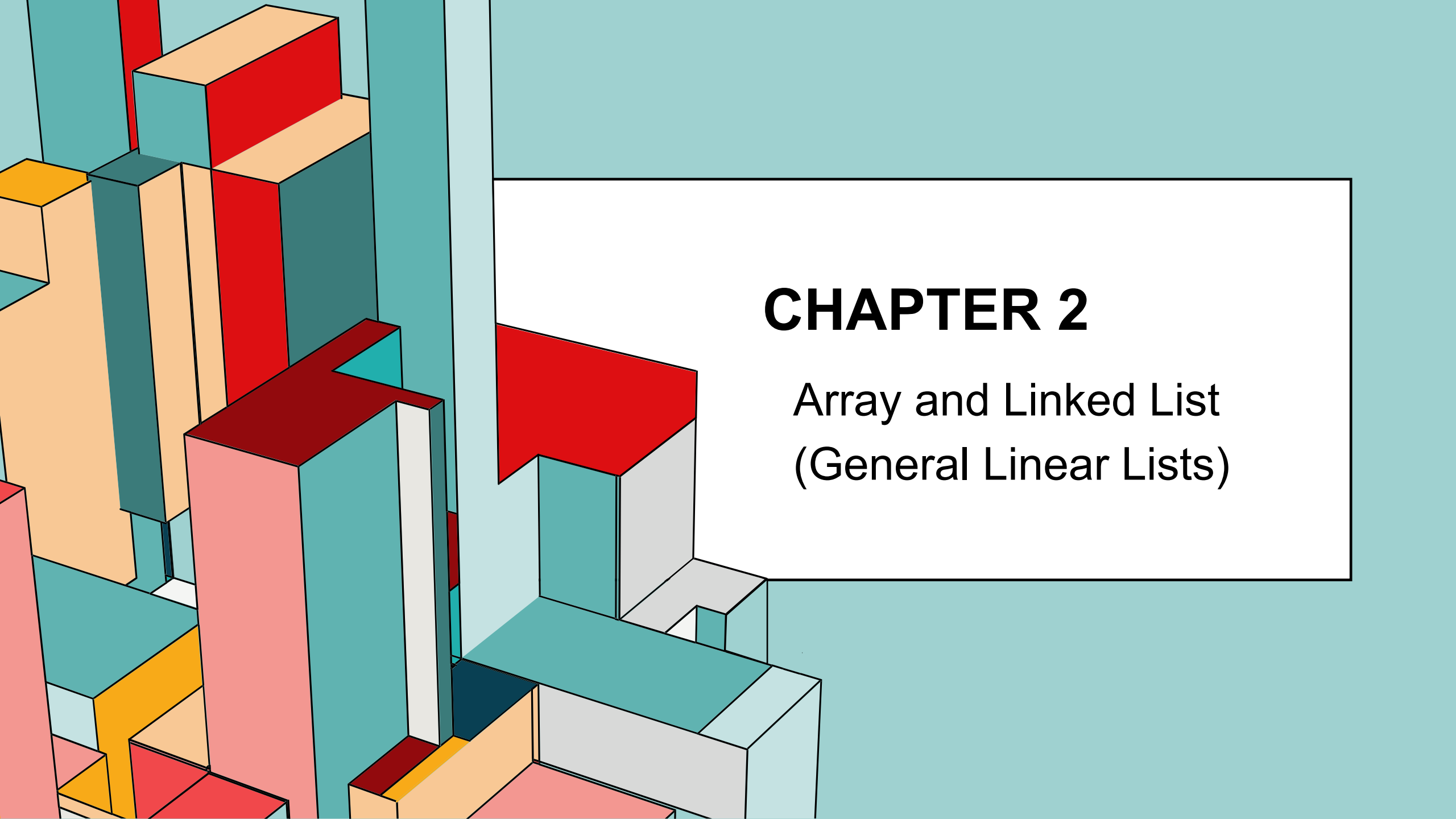


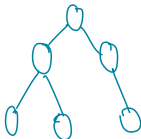


CHAPTER 2

Array and Linked List (General Linear Lists)

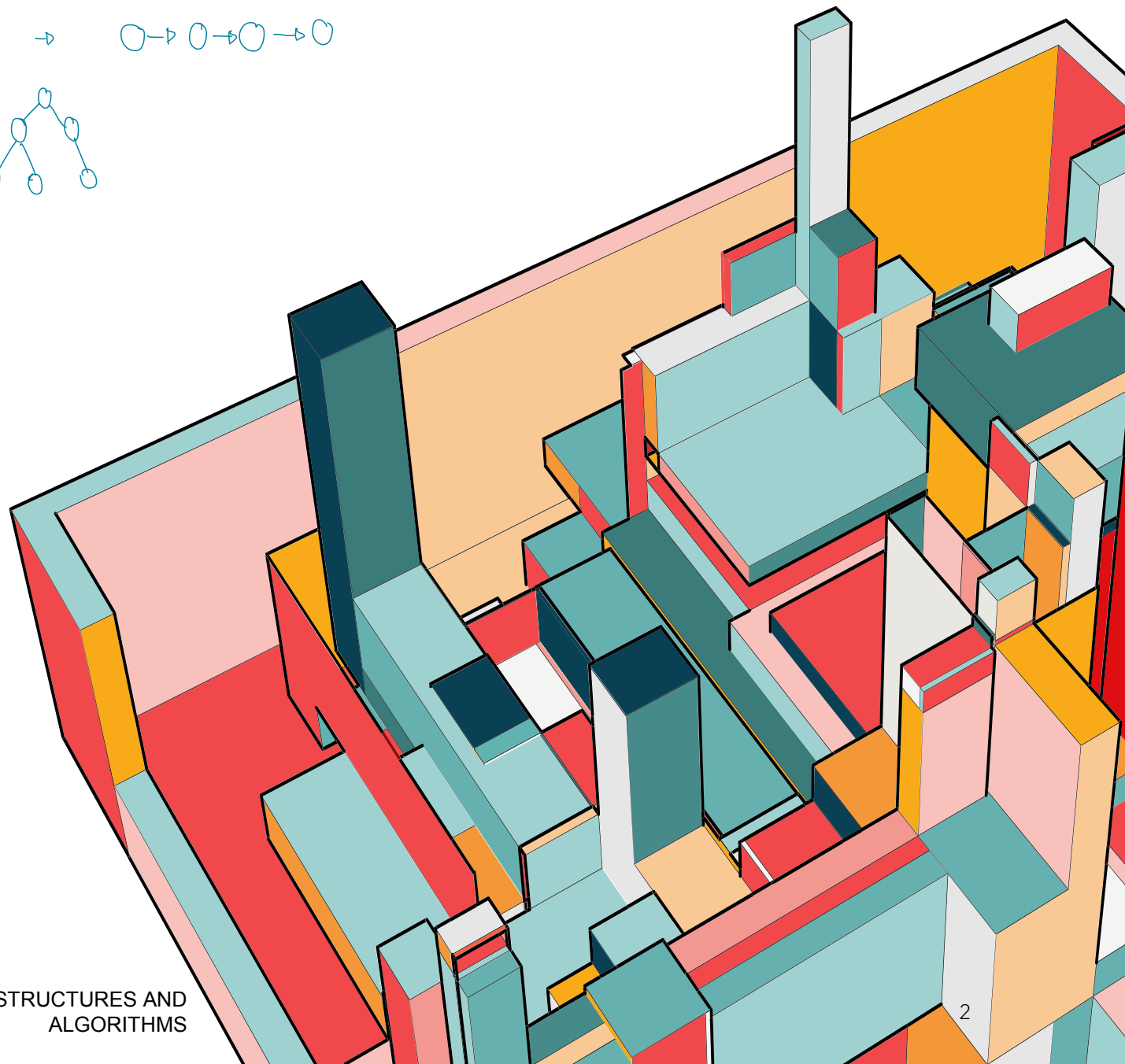
จุดประสงค์

1) Linear List $\rightarrow$ $\bigcirc \rightarrow \bigcirc \rightarrow \bigcirc \rightarrow \bigcirc$

2) Tree $\rightarrow$ 

3) Graph

- ❖ เข้าใจแนวคิดและการทำงานเกี่ยวกับโครงสร้างข้อมูลแบบ Linear List
- ❖ ประยุกต์แนวคิดโครงสร้างข้อมูลแบบ Linear List เข้ากับการเขียนโปรแกรมได้



ตัวอย่างการใช้ LINEAR LIST

❖ เกม

- High Scores
- Map

Score

๙๘	๙๗	๙๖	๙๕	๙๔	๙๓
----	----	----	----	----	----

Name

A	B	C	D	E
---	---	---	---	---

❖ ระบบปฏิบัติการ

- คิวการจัดการงาน

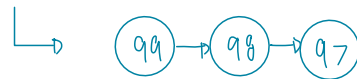
LINEAR LIST

ในการสร้างโครงสร้างข้อมูลแบบลิสต์ มักสร้างโดย

อาร์เรย์ (Array) `int score[5]` →

99	98	97	96	95
----	----	----	----	----

ลิงค์ลิสต์ (Linked List) -> ลิงค์ลิสต์ คือ อะไร ?



MEMORY

ตำแหน่งเก็บข้อมูล

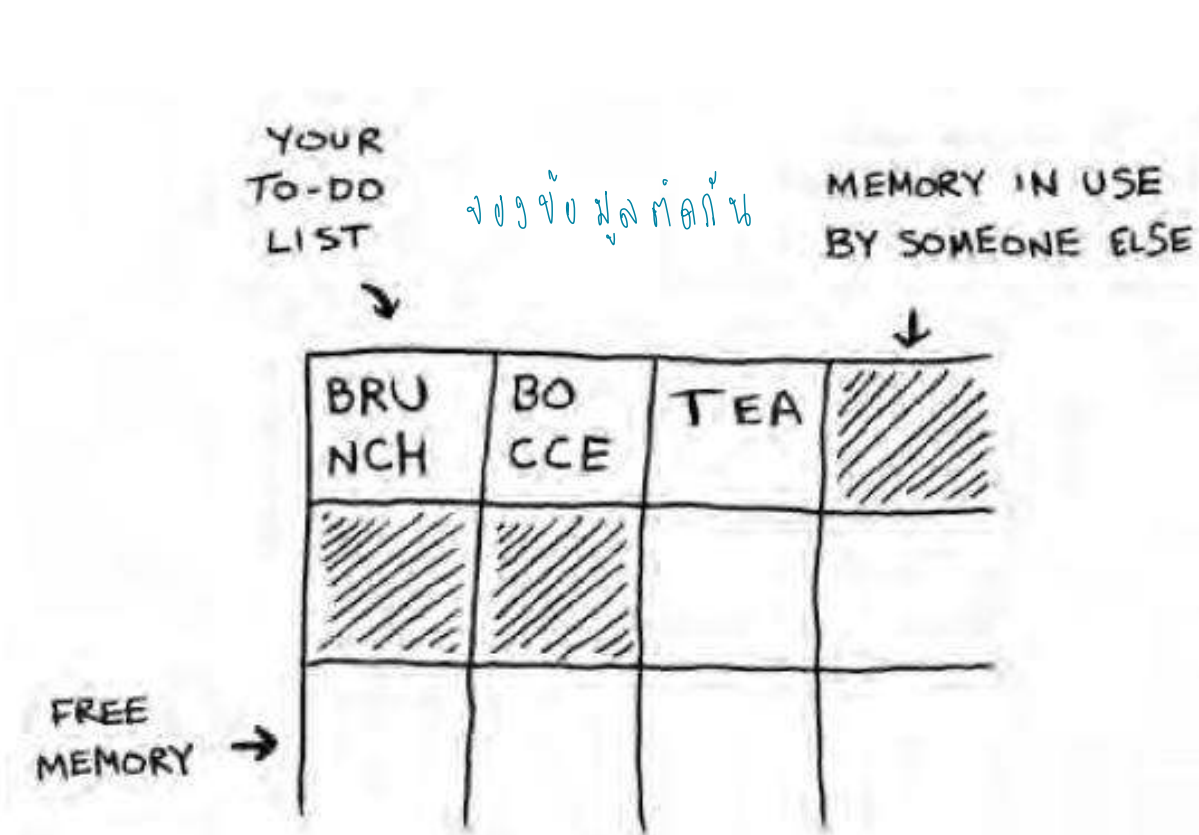
อ้างอิงข้อมูลใน Memory ด้วย Address



ADDRESS: fe0ffeeb

.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.

ARRAY



- ข้อดี 1) เข้าข้อมูล (Access ข้อมูลได้เร็ว)
- ข้อเสีย 1) ขยายไม่ได้ / ลบไม่ได้

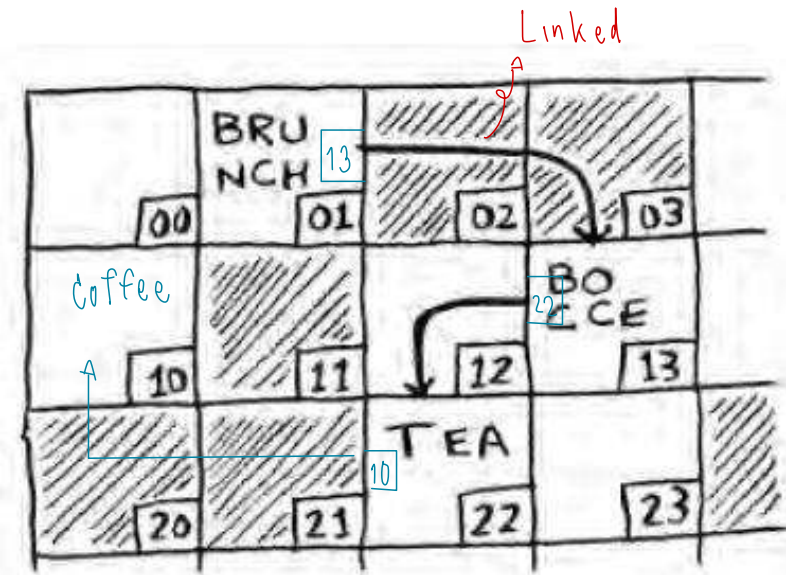
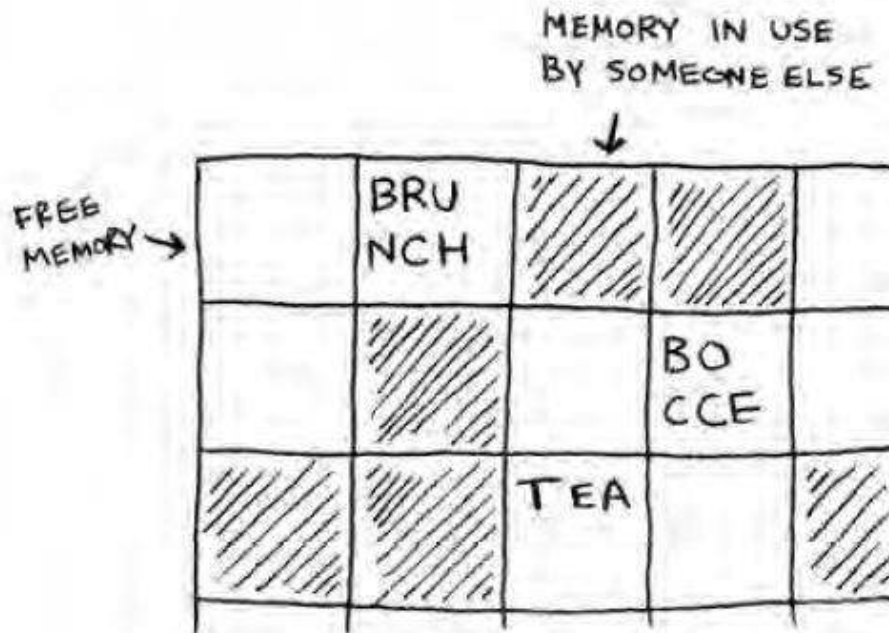
LINKED LIST

ข้อดี

- ง่าย / ขยายข้อมูลได้

ข้อเสีย

- เสียพื้นที่เพิ่ม → เก็บ link coffee
- Address ช้า → วิ่งตาม link ไปเรื่อย ๆ



Static & Dynamic Allocation

❖ Static Memory Allocation (ก. จอง พก. ของ Array)

- ในช่วงของการ Compile เครื่องได้จัดสรรหน่วยความจำ ให้กับตัวแปรแต่ละตัวล่วงหน้า ในขนาดที่คงที่
- เมื่อมีการประมวลผล เนื้อที่เหล่านี้ไม่สามารถขยายหรือลดลงได้ (เช่น Array)

จอง พก. แบบ fix ไว้ล่วงหน้า เพิ่ม/ลด ไม่ได้

Static & Dynamic Allocation

❖ Dynamic Memory Allocation

- สามารถกำหนดตัวแปรไว้ก่อน แต่ยังไม่ต้องจัดสรรเนื้อที่หน่วยความจำจนกว่าจะถึงช่วง **run time** (ตอนประมวลผลโปรแกรม)
- สามารถสร้างตัวแปรขึ้นได้ทุกครั้งที่ต้องการใช้ และสามารถทำลายลงเมื่อไม่ต้องการ
- มีความยืดหยุ่น
↳ เพิ่มได้ลดได้ง่าย ๆ
- ตัวแปรที่เป็นแบบ Dynamic นี้ได้แก่ Pointer และ Linked List

* สิ่งอื่น ๆ

ARRAY

- ตัวแปรที่เก็บชุดข้อมูลซึ่งมีชนิดเดียวกัน โดยทำการจองพื้นที่ในหน่วยความจำต่อเนื่องกัน
- ตัวอย่างการใช้ Array ในภาษา C
 ประกาศ Array S ง่าย
 - `int score[5] = {10, 20, 30, 40, 50}`
 - `score[0] -> 10`
 - `score[4] -> 50`

	index				
	0	1	2	3	4
score	10	20	30	40	50

LINKED LIST

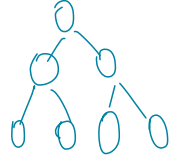
❖ Singly Linked List $\bigcirc \rightarrow \bigcirc \rightarrow \bigcirc \rightarrow \bigcirc$

- ลิงค์ลิสต์ที่มีลิงค์ชี้ไปในทิศทางเดียว

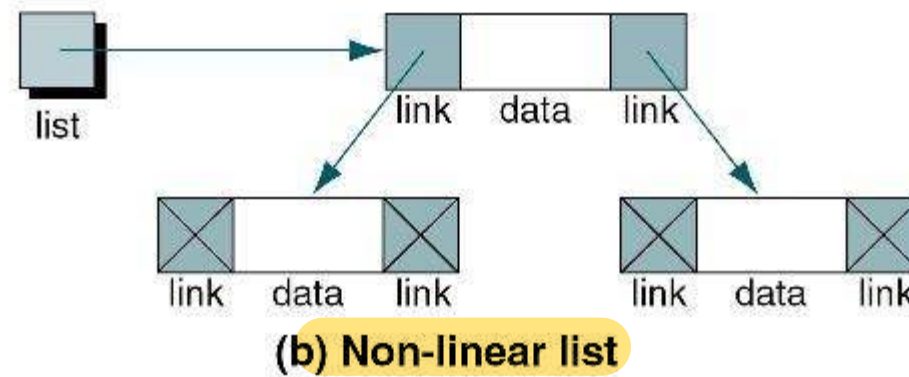
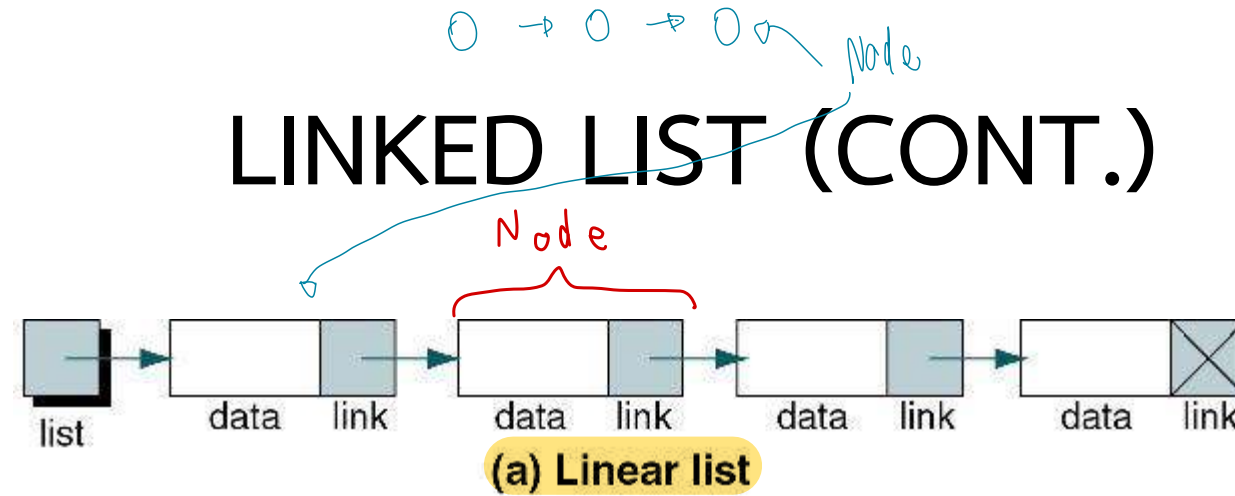
❖ Doubly Linked List $\bigcirc \rightleftarrows \bigcirc \rightleftarrows \bigcirc$

- ลิงค์ลิสต์ที่มีลิงค์ชี้ในสองทิศทาง (ไป-กลับ)

LINKED LIST

- ❖ ลิงค์ลิสต์ : โครงสร้างข้อมูลทีประกอบด้วยกลุ่มโหนดที่มีการเรียงลำดับ
- ❖ เรานำลิงค์ลิสต์มาใช้ในการสร้าง
 - *Linear structures*
 - *Non-linear structures* → Tree  , etc.

LINKED LIST (CONT.)

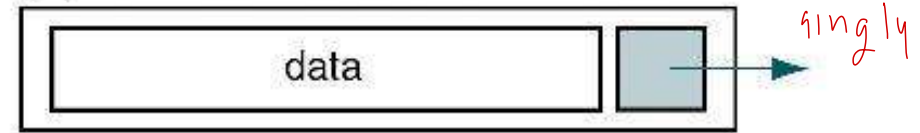


NODES

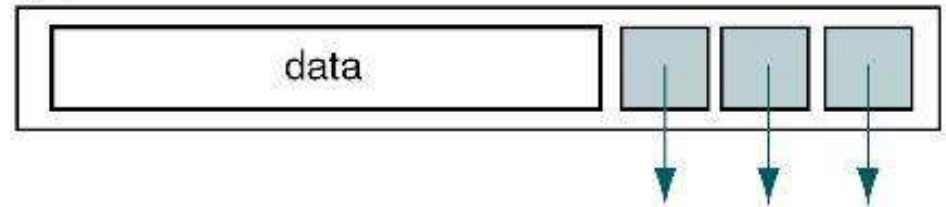
❖ โหนด เป็นโครงสร้างที่ประกอบด้วย

- ข้อมูล (Data)
- ลิงค์ (Link)

(a) Node in a linear list

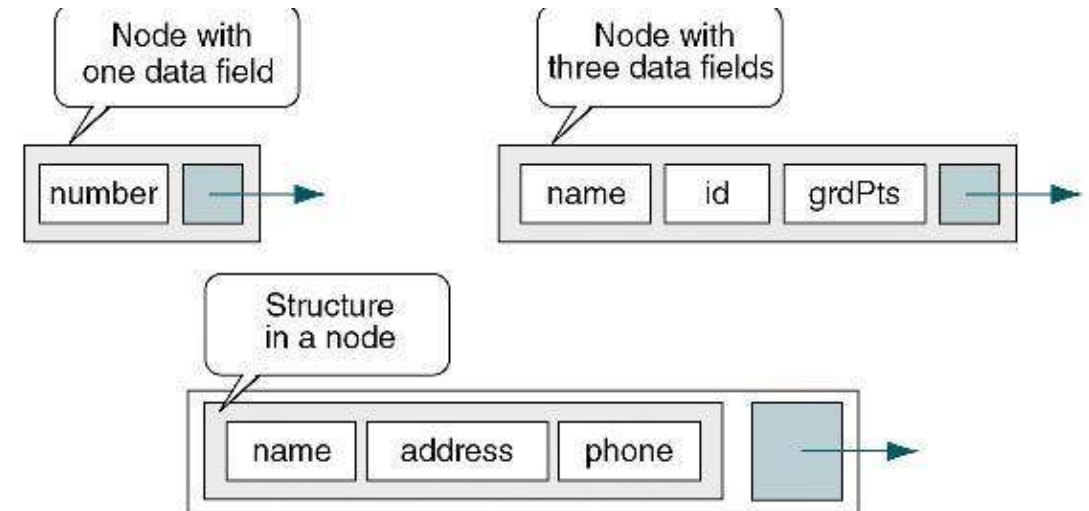


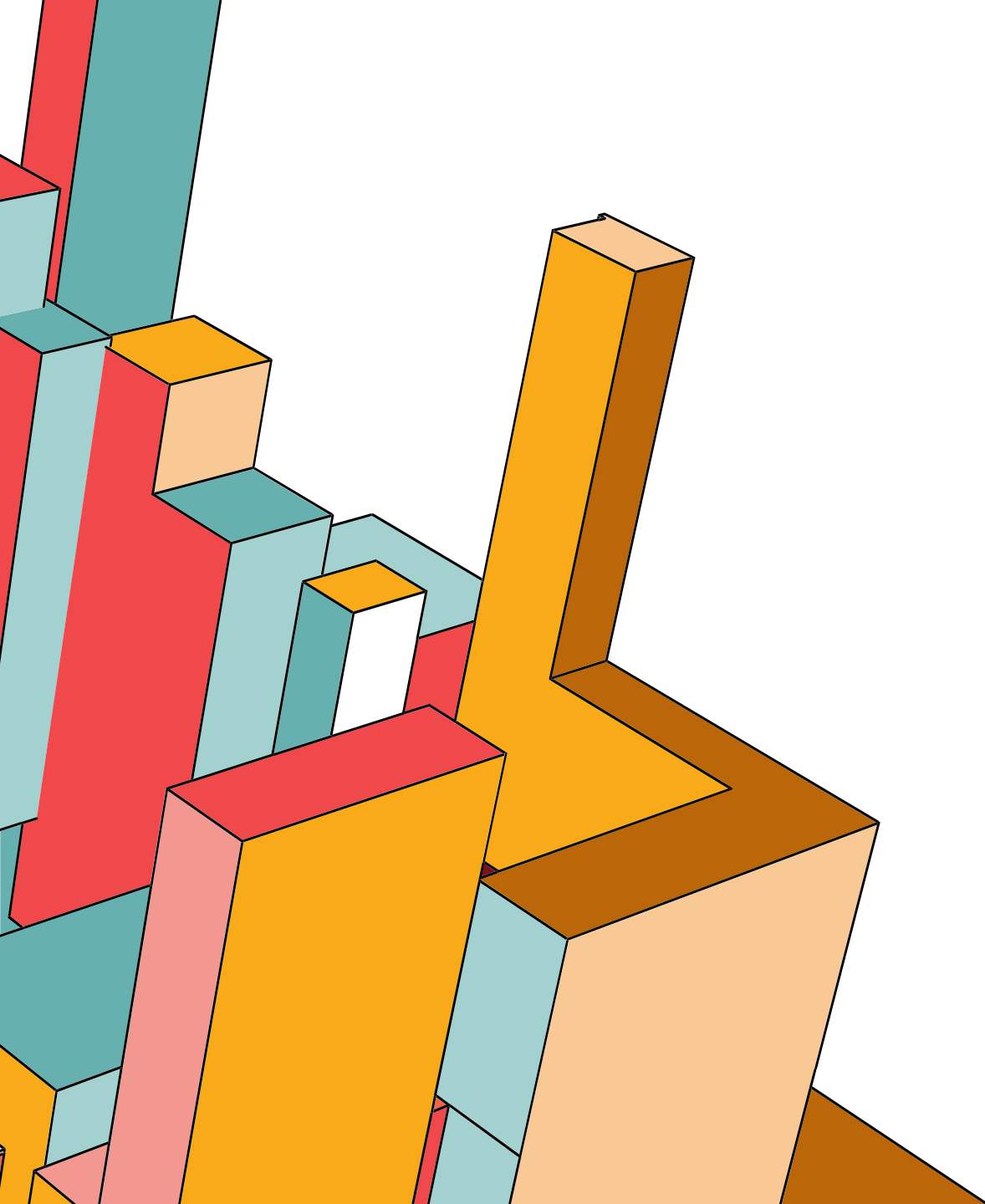
(b) Node in a non-linear list



LINKED LIST NODE STRUCTURES

- ❖ ข้อมูลของโหนด อาจจะประกอบด้วย
- ข้อมูลชุดเดียว
 - ข้อมูลหลายชุด
 - ข้อมูลหนึ่งชุดที่ประกอบด้วยข้อมูลย่อยหลายชุด



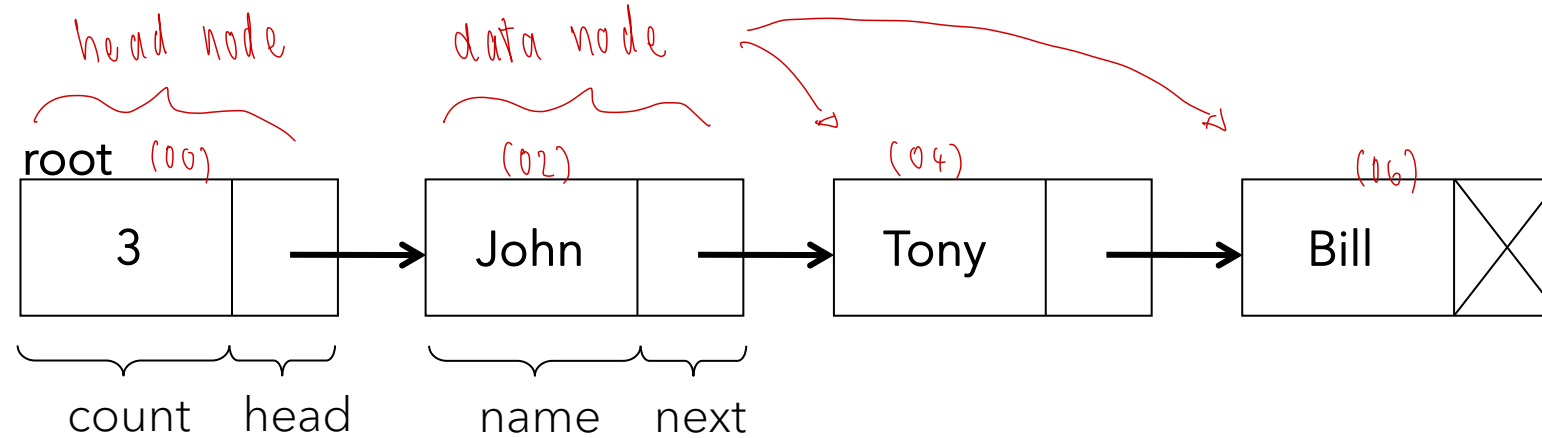


LINKS

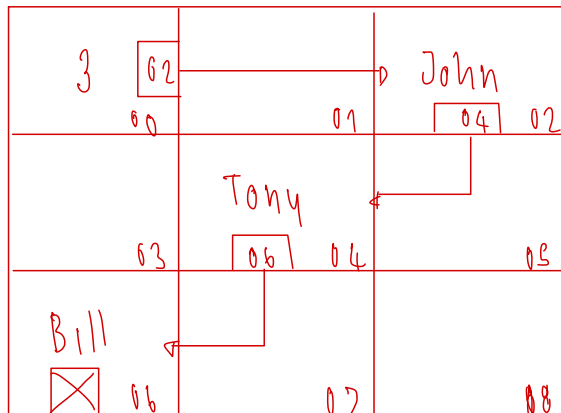
- ❖ ลิงค์ของโหนด จะทำหน้าที่เชื่อมโหนดนั้นกับโหนดอื่นเข้าด้วยกัน
- ❖ เราสามารถนำพอยน์เตอร์ (pointer) มาช่วยในการสร้างลิงค์ได้
- ❖ พอยน์เตอร์ คือ ตัวชี้ไปยังข้อมูล
 - นั่นก็คือ ตัวเก็บตำแหน่ง (address) ของข้อมูลนั่นเอง

Node → head node → meta data (ขนาดของข้อมูล)
 → data node

SINGLY LINKED LIST



root.count → 3
 root.head.name → 'John'
 root.head.next.name → 'Bill'
 root.head.next.next → Null
 root.next.next.name → error



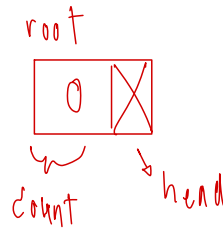
เราจะสร้างลิงค์ลิสต์นี้ได้อย่างไร -> ต้องมีการกำหนดโครงสร้างของ
 โหนดก่อน

☐ ข้อมูล -> ชื่อ

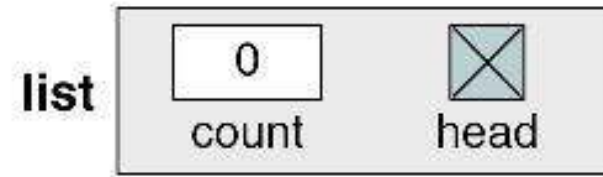
☐ ลิงค์ -> ชี้ไปยังโหนดถัดไป

CREATE LIST

create list(root)



head = pointer ที่ชี้ไปที่ data node



Head_node ← ปกติจะเก็บใน struct
int count
Data_node head
end Head_node

head node → เก็บข้อมูล

data node →
 เก็บข้อมูล
 ใน list

- Head Node : กำหนดข้อมูล
 รายละเอียดเกี่ยวกับลิสต์

```
Algorithm createList (list)
Initializes metadata for list.
    Pre    list is metadata structure passed by reference
    Post   metadata initialized
1 allocate (list)
2 set list head to null
3 set list count to 0
end createList
```

NODE STRUCTURE

Data_node

string name

Data_node next

end Data_node

Head_node

int count

Data_node head

end Head_node

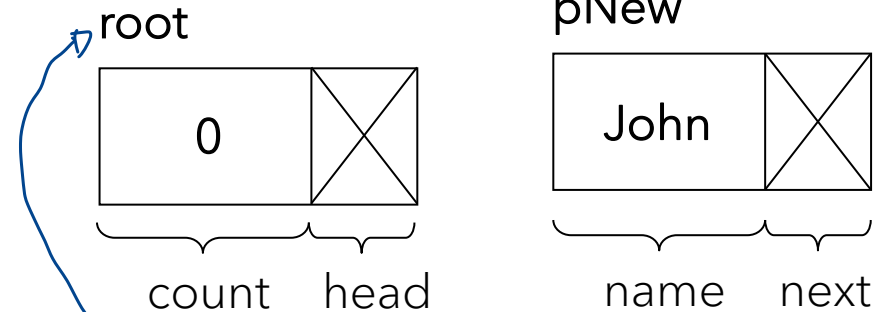
สร้าง DataNode

สร้าง head Node

Ex. Create Tony, NULL



Create DataNode('John, NULL)



create data node

```
createList(root)
createDataNode (d,p):
    Data Node
    pNew = allocate(Data_node)
    name = d
    next = p
    return pNew
End createDataNode
```

BASIC OPERATIONS

- Insertion
- Deletion
- Retrieval *การดึงข้อมูลออกมา*
- Traversal *การเข้าถึงข้อมูล*

INSERTION

- แทรกโหนดที่ส่วนหัวของลิสต์
- แทรกโหนดที่ส่วนท้ายของลิสต์
- แทรกโหนดตามการเรียงลำดับข้อมูล

createList(root)

createDataNode('John', NULL)

root.head = pNew

root.count += 1

createDataNode('Tony', NULL)

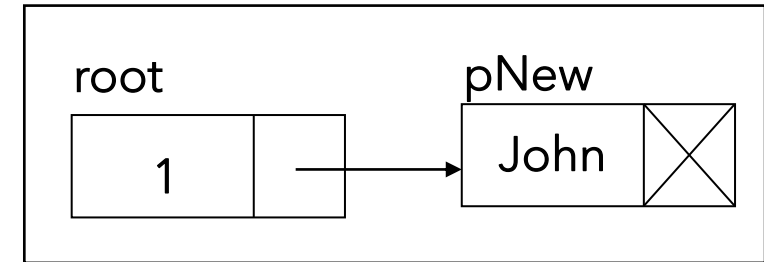
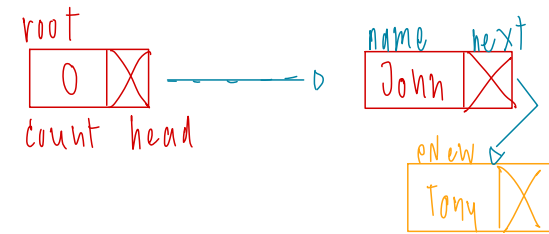
- เพิ่มโหนดที่เก็บชื่อ John เข้าไปในลิสต์

- Set root head to pNew
- Increase root.count

INSERTION

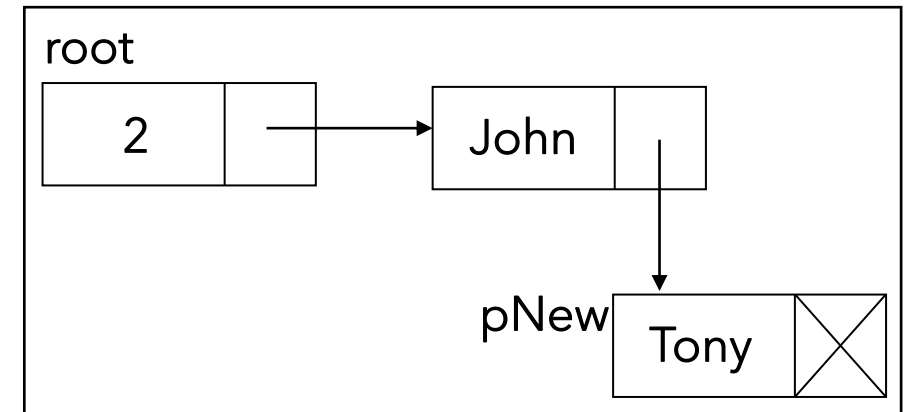
root.head.next = pNew

root.count += 1



- อยากจะเพิ่มโหนดที่เก็บชื่อ Tony ต่อท้ายเข้าไป ไม่ยากเลย

- createDataNode(Tony, null)
- Set next of last node to pNew
- Increase root count



INSERTING NODES AT THE FRONT OF A LIST

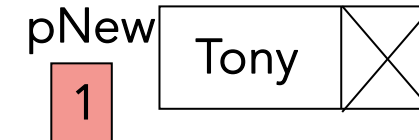
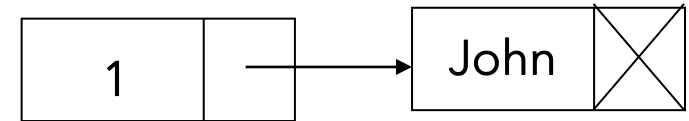
1

❖ ถ้าจะเพิ่มโหนดที่มีชื่อเป็น Tony ไว้ต้นลิสต์

- *createDataNode(Tony,null)*

- $pNew.next = root.head$
- $root.head = pNew$
- Increase root count

root

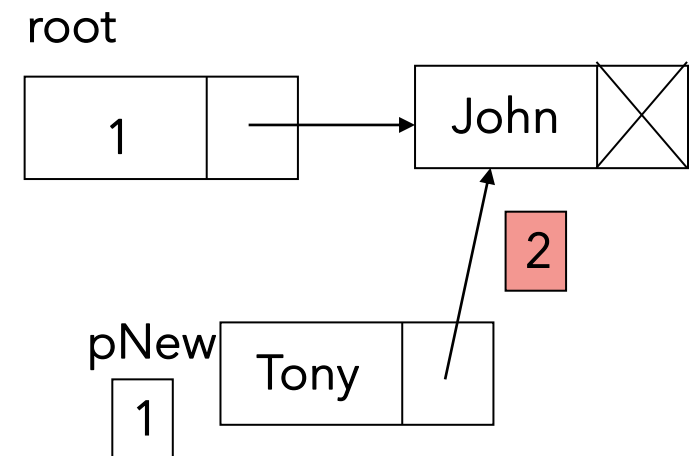


INSERTING NODES AT THE FRONT OF A LIST

2

❖ ถ้าจะเพิ่มโหนดที่มีชื่อเป็น Tony ไว้ต้นลิสต์

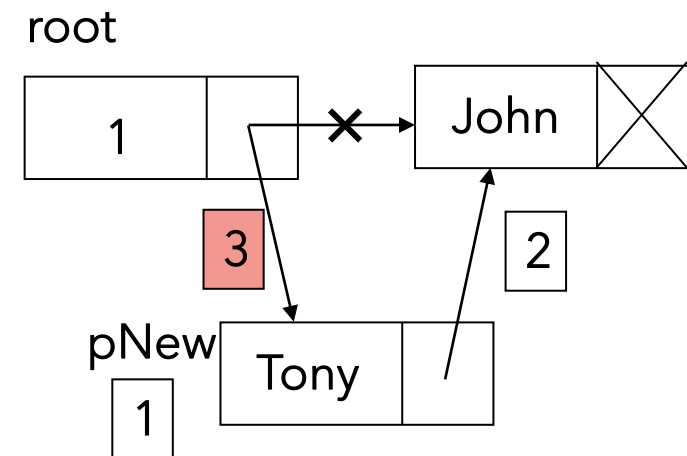
- createDataNode(Tony,null)
- *pNew.next = root.head*
- root.head = pNew
- Increase root count



INSERTING NODES AT THE FRONT OF A LIST

❖ ถ้าจะเพิ่มโหนดที่มีชื่อเป็น Tony ไว้ต้นลิสต์

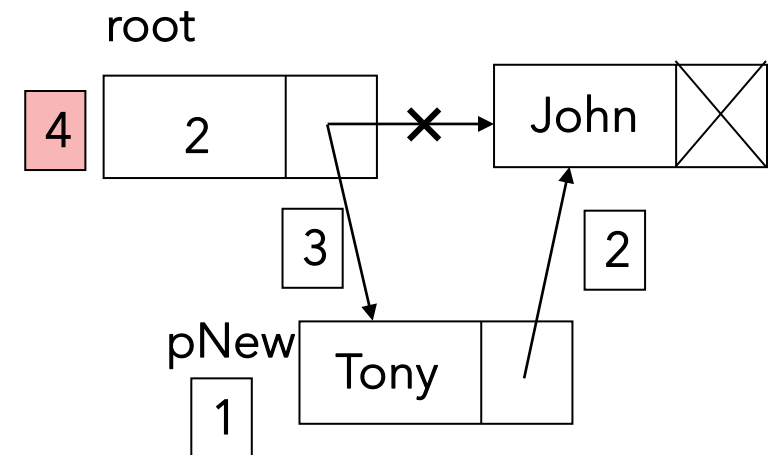
- createDataNode(Tony,null)
- pNew.next = root.head
- root.head = pNew
- Increase root count



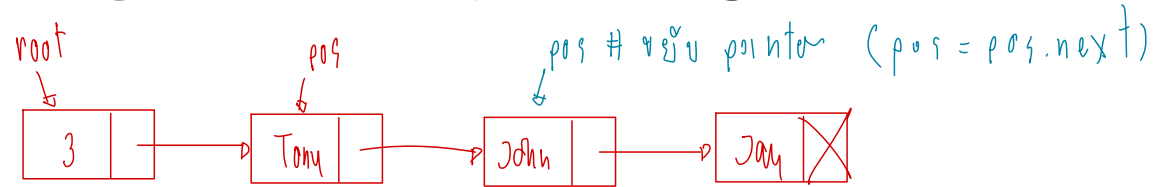
INSERTING NODES AT THE FRONT OF A LIST

❖ ถ้าจะเพิ่มโหนดที่มีชื่อเป็น Tony ไว้ต้นลิสต์

- createDataNode(Tony,null)
- pNew.next = root.head
- root.head = pNew
- Increase root count



TRAVERSING A LINKED LIST



Algorithm printList(List root)

~~Data Node Person~~ pos

pos = root.head

loop (pos != null)

print pos.name # print Tony

pos = pos.next # รับ pointer

end loop

end printList

- เป็นการเข้าถึงโครงสร้างทีละโครงสร้างที่อยู่ในลิสต์นั้น
- ตัวอย่างการพิมพ์ค่าที่อยู่ใน Linked List

INSERTING NODES AT THE END OF A LIST

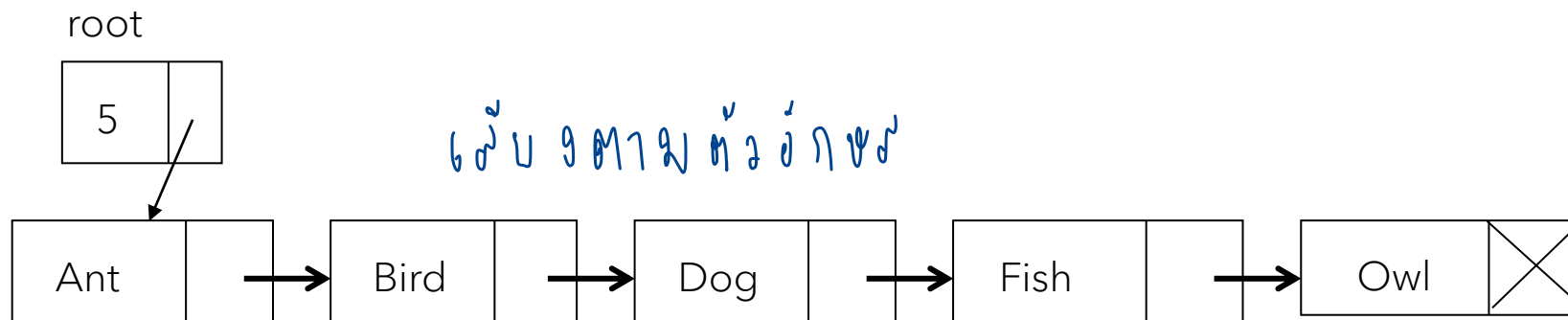
❖ เหมือนกับตัวอย่างก่อนหน้านี้ ปัญหาคือ เรารู้ได้อย่างไรว่าไหนไหนเป็นไหนท้ายของลิสต์

- ต้องท่อง (Traverse) เข้าไปในลิสต์จนถึงส่วนท้ายสุด

```
pNew = createDataNode(Tony,null)  # สร้าง Node ใหม่
If (root.head is null)             # empty list
    root.head = pNew
else
    start = root.head
    loop (start.next != null)      # check ว่า เป็นที่สิ้นสุดท้าย
        start = start.next
    end loop                       # จบ loop
    start.next = pNew              # เชื่อมไว้ที่ตัวสุดท้าย
end if
```

INSERTING NODES INTO AN ORDERED LIST

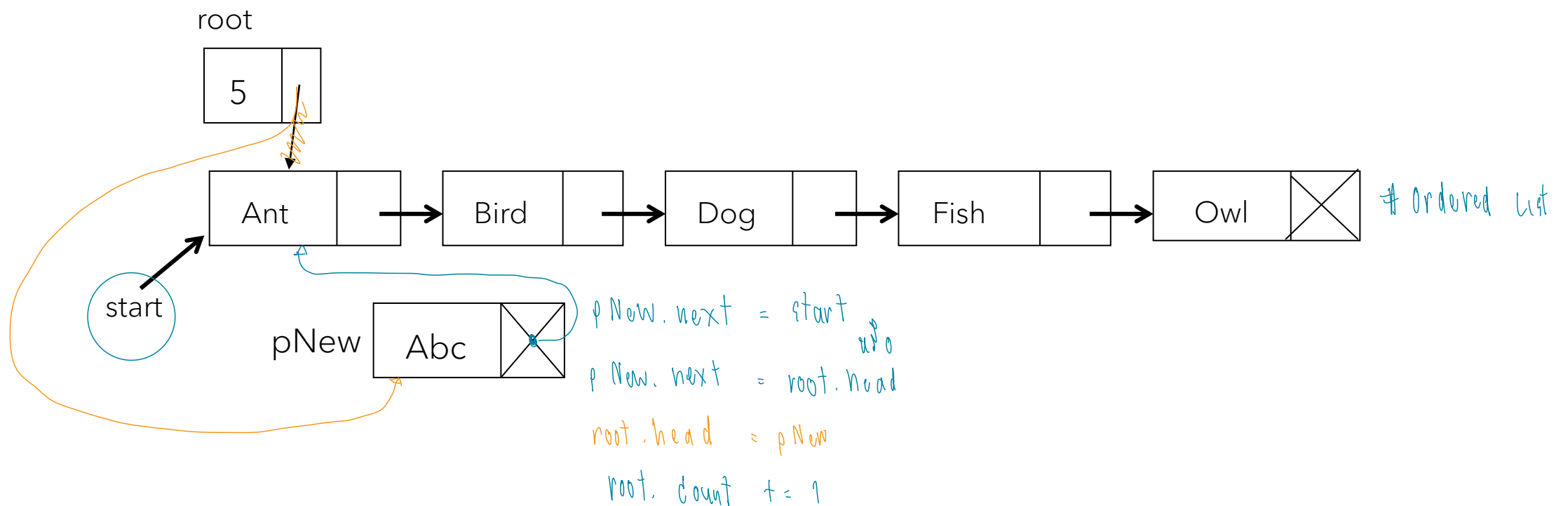
- แทรกโหนดใหม่ โดยมีการเรียงลำดับข้อมูล
- ปัญหา คือ จะไปแทรกตรงไหน
 - ต้องท่องเข้าไปในลิสต์เรื่อยๆ ขณะเดียวกันก็ต้องเปรียบเทียบค่าข้อมูลไปด้วย



INSERTING NODES INTO AN ORDERED LIST

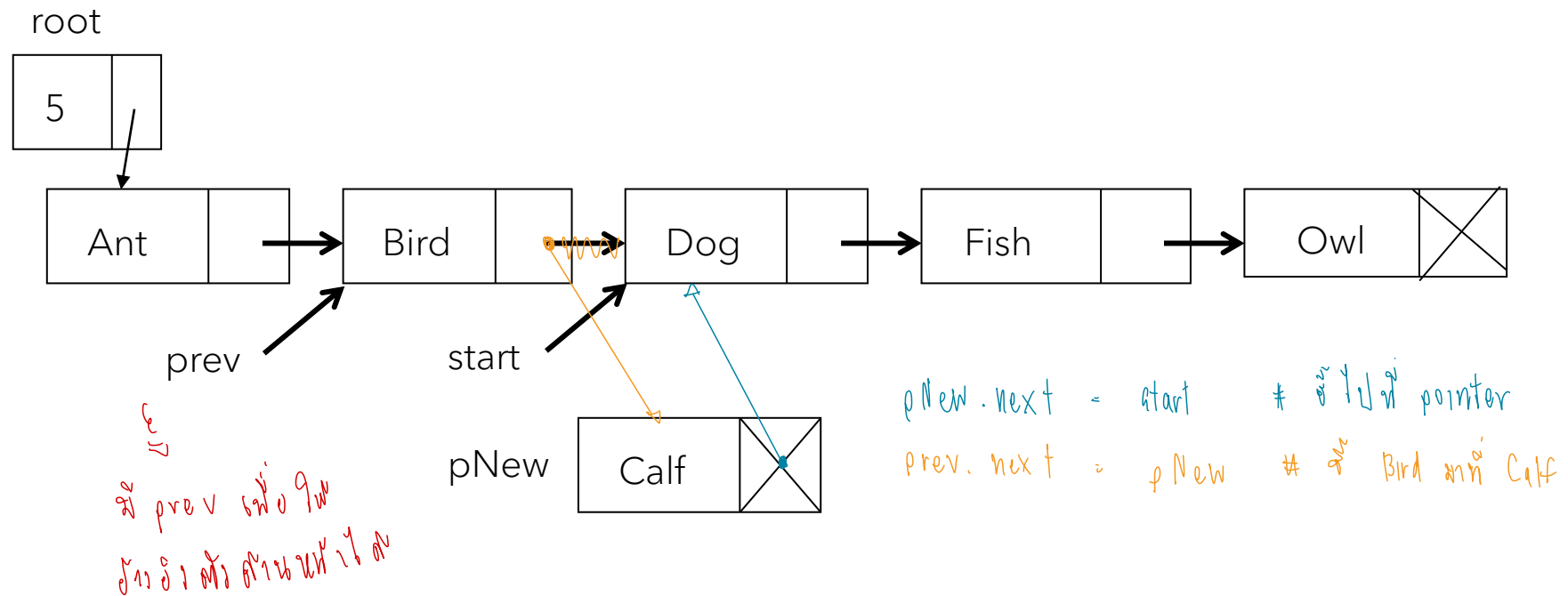
Insert Node 9 hai $\rightarrow$ pointer start

- กรณิแหทรกโหนดตานหนาสุดของลิสต์



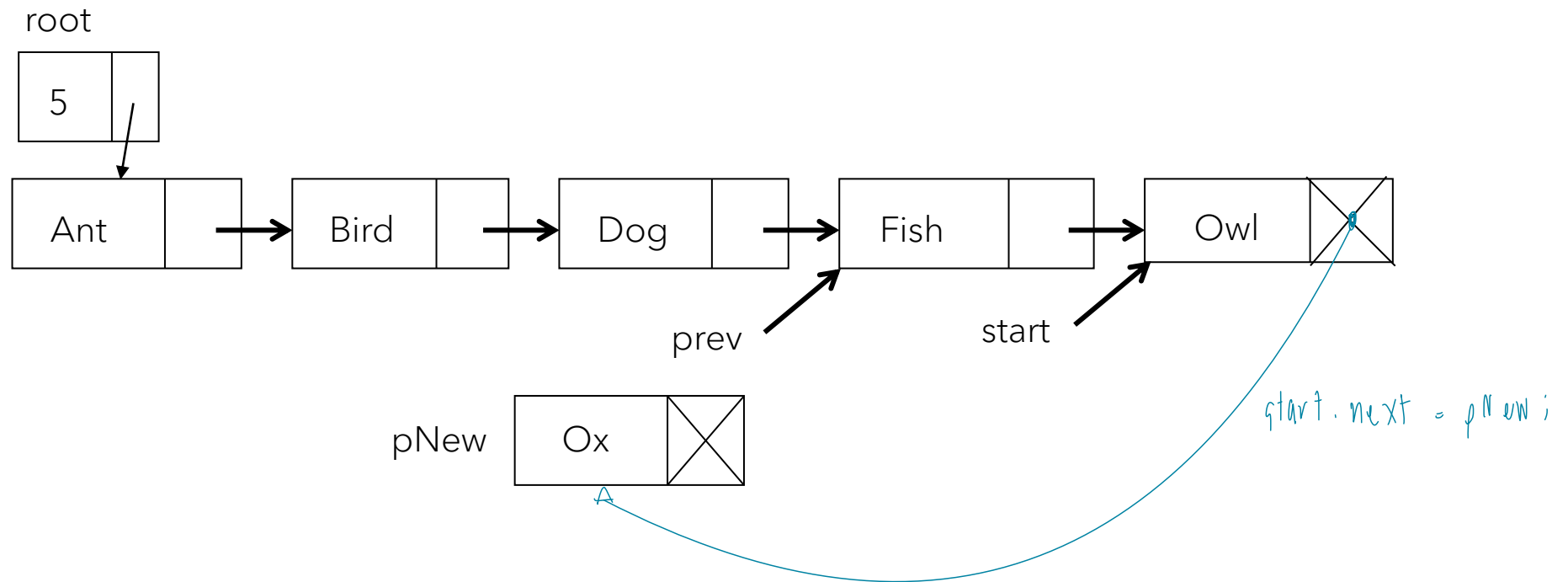
INSERTING NODES INTO AN ORDERED LIST

- กรณีแทรกกระหว่างโหนดในลิสต์



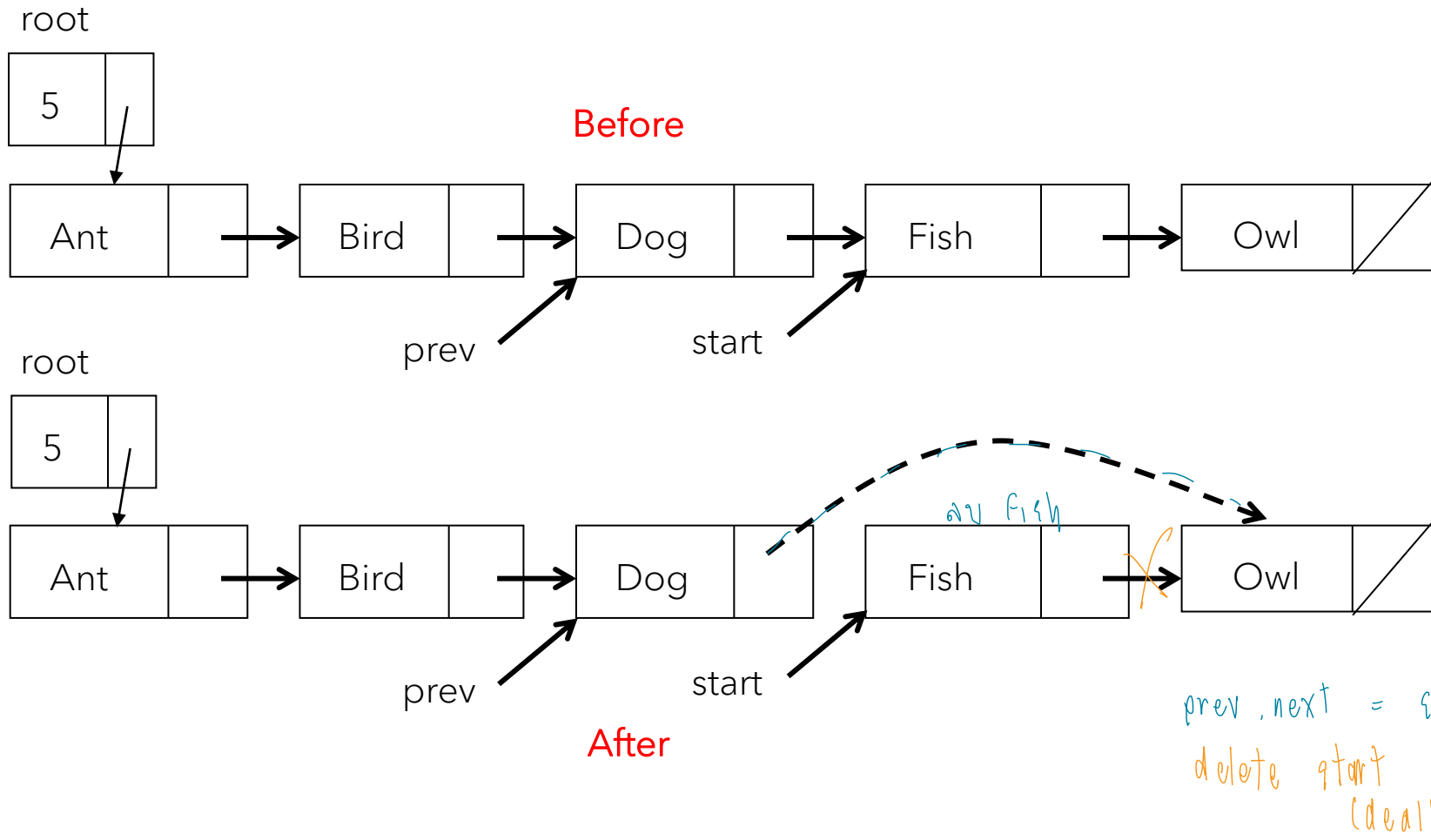
INSERTING NODES INTO AN ORDERED LIST

- กรณีที่แทรกปลายลิสต์



DELETE

start ၌ Node ကိုရှာရ



DELETE

- ในกรณีที่โหนดที่ต้องการลบเป็นโหนดแรกของลิสต์
`prev.next = start.next` Error

DELETING NODES FROM AN UNORDERED LIST

- กรณีที่ต้องการลบโหนดที่อยู่หัวลิสต์

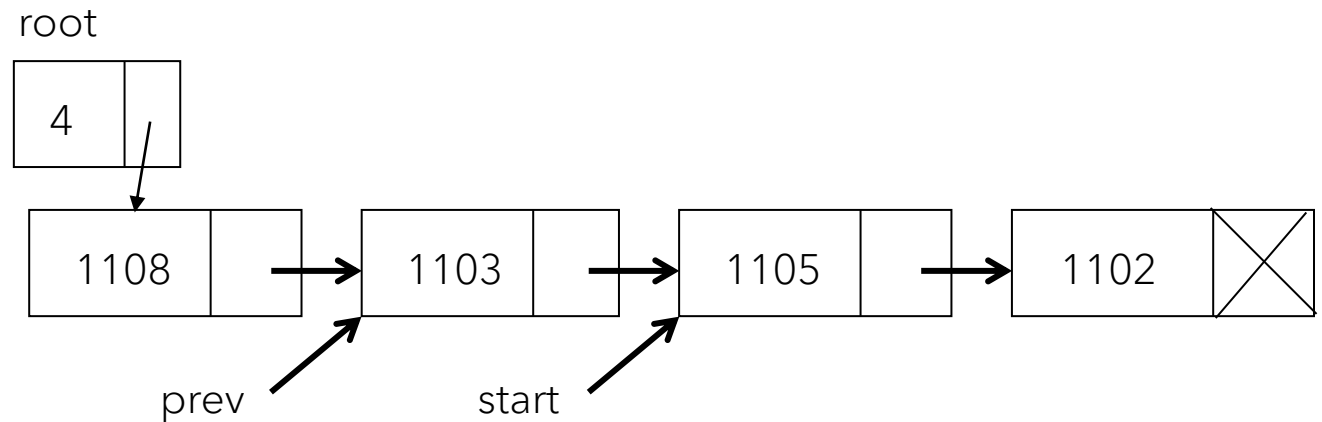
`root.head = root.head.next ; delete start;`

- กรณีที่ต้องการลบโหนดที่อยู่ระหว่างลิสต์

`prev.next = start.next; delete start;`

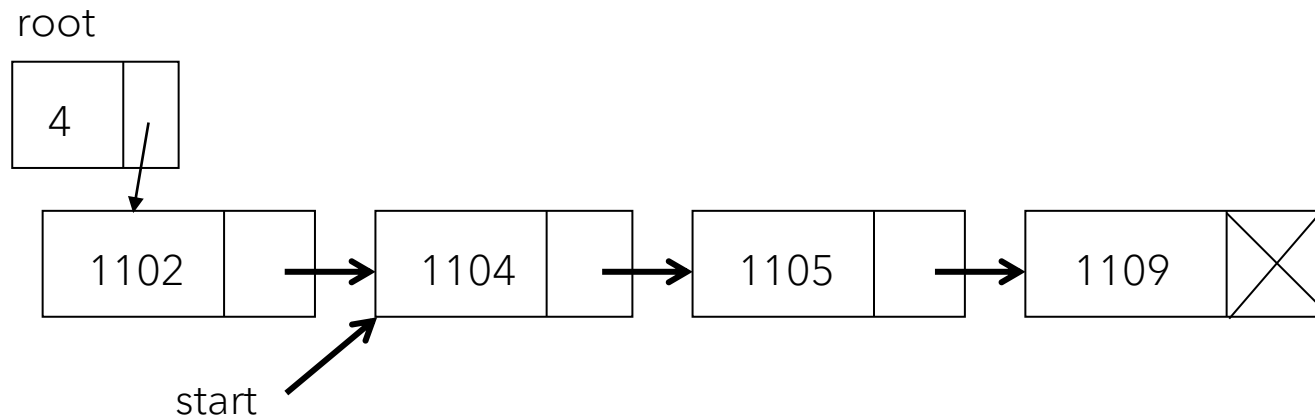
- กรณีที่ต้องการลบโหนดที่อยู่ท้ายลิสต์

`prev.next = NULL; delete start;`



DELETING NODES FROM AN ORDERED LIST

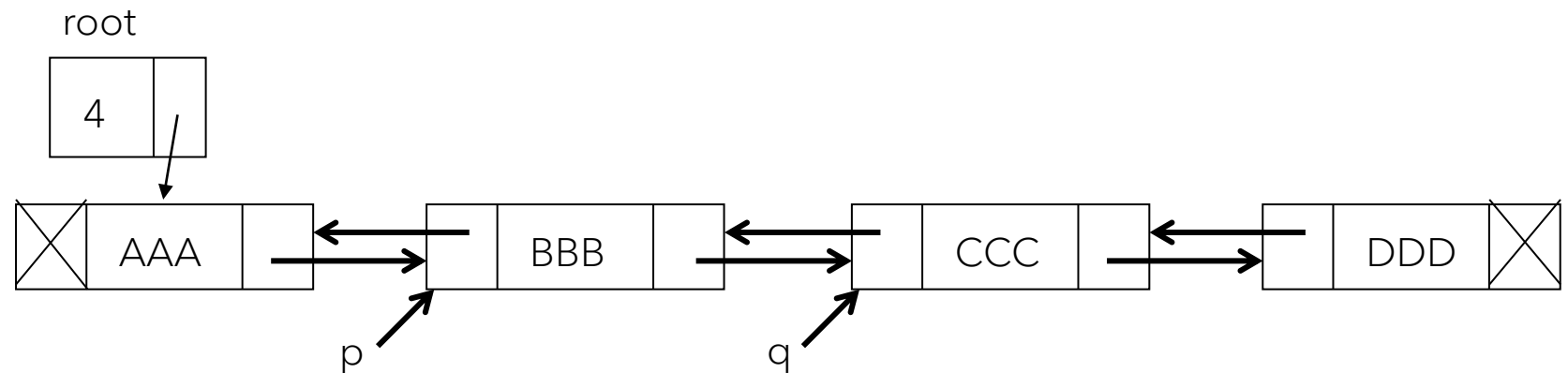
- ท่องลิสต์ไปเรื่อยๆ เมื่อพบโหนดที่ต้องการลบก็ลบไป
 - ปัญหา คือ ถ้าไม่มีโหนดที่ต้องการลบในลิสต์ จะต้องทำการท่องลิสต์จนหมด
 - สามารถทราบว่าโหนดนั้นไม่มีในลิสต์ แม้ว่าจะยังท่องลิสต์ไม่หมด
 - เช่น ถ้าต้องการลบโหนดที่เก็บเลข 1103 เมื่อท่องลิสต์จนไปถึงโหนดที่เก็บเลข 1104 แล้วยังไม่พบโหนดที่ต้องการ สรุปลงได้ว่า ไม่มีโหนดที่ต้องการลบ



DOUBLY LINKED LIST

- เป็นลิงค์ลิสต์ที่มีลิงค์ 2 ทิศทาง คือ จากหน้าไปหลัง และ หลังไปหน้า

q.back.next.next.data = DDD



node

type data

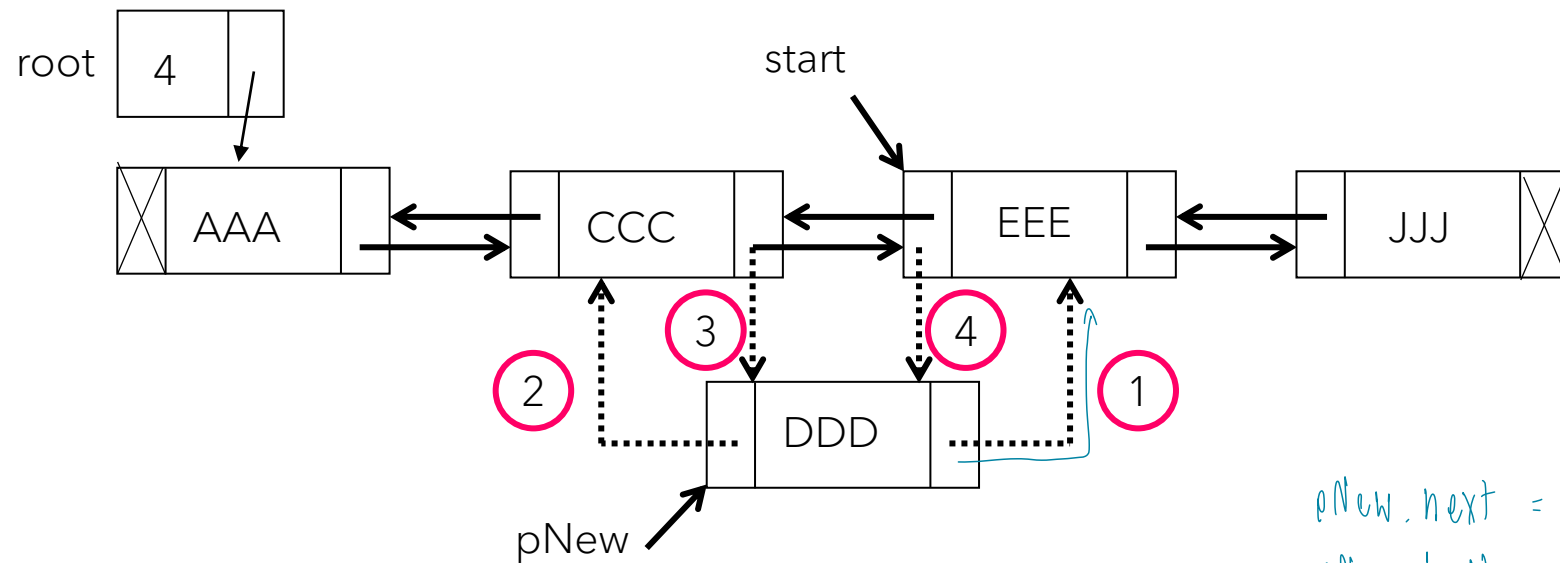
node back

node next

end node



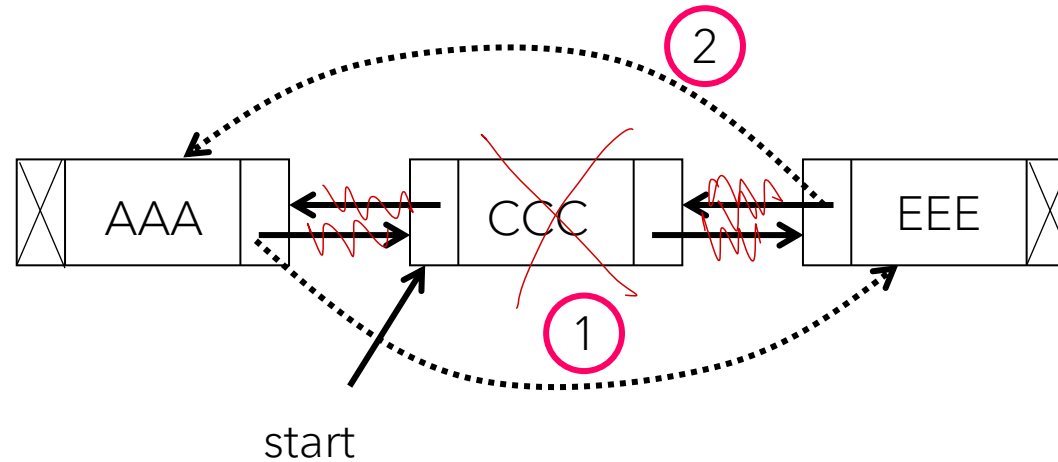
INSERTING NODES INTO A DOUBLY LINKED LIST



ต้องการแทรกโหนด pNew ไว้หน้า start

```
pNew.next = start  
pNew.back = start.back  
start.back.next = pNew  
start.back = pNew  
root.count += 1
```

DELETING NODES FROM A DOUBLY LINKED LIST



- ต้องการลบโหนด start

```
start.back.next = start.next  
start.next.back = start.back  
delete start
```

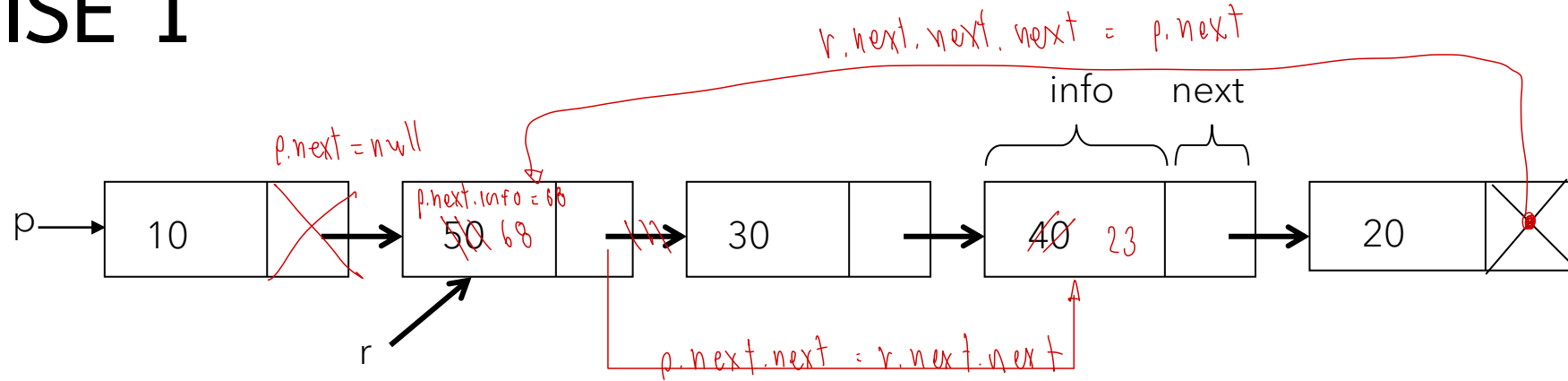
LIST IN PYTHON

❖ ตัวอย่างการใช้ List ใน Python

- `my_list = [10, True, 2.5, 50, False]`
- `my_list[0] -> 10`
- `my_list.insert(0, 15.5)`
- `del my_list[1]`

❖ ลิสต์ในไพธอน ไม่ใช่ลิงค์ลิสต์ แต่เป็นอาร์เรย์ของพอยน์เตอร์ (แอดเดรส) ที่ชี้ไปที่วัตถุต่างๆ

EXERCISE 1



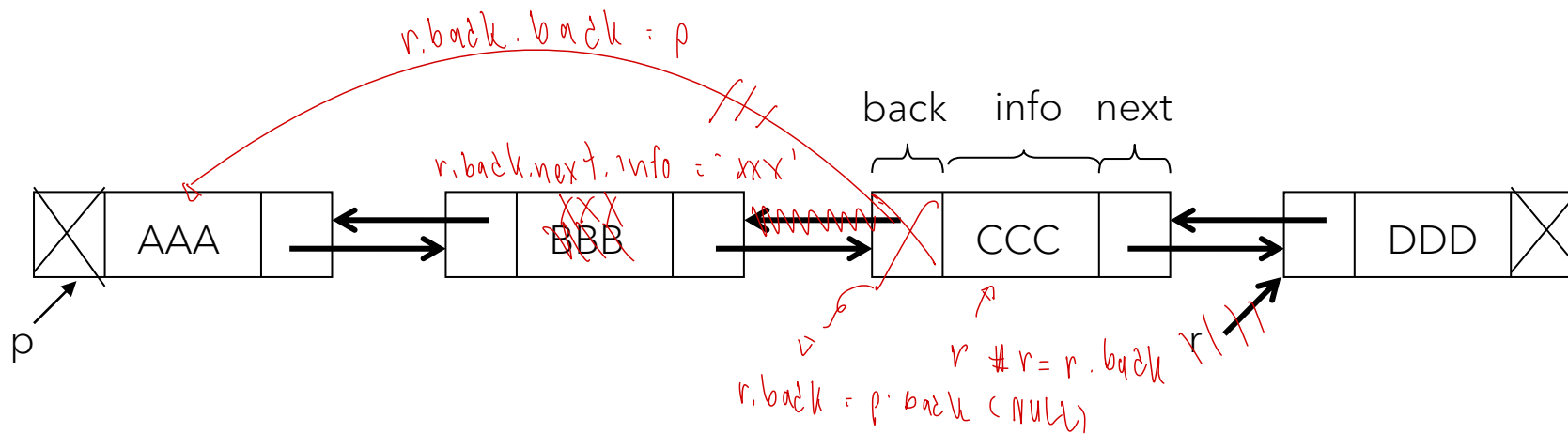
- จงประมวลผลคำสั่งต่อไปนี้ พร้อมวาดภาพผลลัพธ์ของแต่ละคำสั่ง
 - $r.next.next.info = 23$
 - $p.next.next = r.next.next$
 - $p.next.info = 68$
 - $r.next.next.next = p.next$
 - $p.next = null$

EXERCISE 2

- จากผลลัพธ์ในข้อก่อนหน้านี้ จงหาผลลัพธ์ของ
loop (r.next != null)
print r.info
r = r.next
end loop

EXERCISE 3

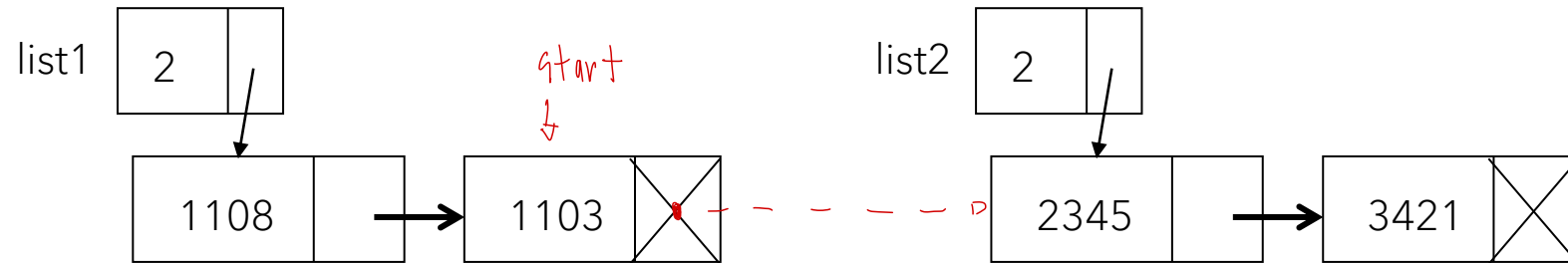
doubly linked list



- จงประมวลผลคำสั่งต่อไปนี้ พร้อมวาดภาพผลลัพธ์ของแต่ละคำสั่ง
 - $r.back.back = p$
 - $r = r.back$
 - $r.back.next.info = 'XXX'$
 - $r.back = p.back$

EXERCISE 4

- จงเขียนอัลกอริทึม append ที่ใช้สำหรับต่อลิสต์ 2 ลิสต์เข้าด้วยกัน
- เช่น `append(list1, list2) -> list1` และ list2 มีขนาดและข้อมูลเป็นเท่าไรก็ได้



- ตัวอย่างผลลัพธ์

